

ShelfScan — Reporte Final

Sistema de Auditoría Visual de Lineales de Supermercado

Curso: Visión por Computadora **Grupo:** Diego Valenzuela · Daniel Dubón · Bianca Calderón
Catedrático: Alberto Suriano **Fecha:** Mayo 2026

1. Descripción del Problema

Los supermercados enfrentan un problema operativo constante: mantener sus estantes correctamente abastecidos y organizados. Un quiebre de stock — un espacio vacío donde debería haber un producto — representa venta perdida directa. Según estudios del sector retail, los quiebres de stock generan pérdidas de entre 4% y 8% de ventas anuales.

El proceso tradicional de auditoría de estantes es manual: un empleado recorre el supermercado, revisa cada lineal visualmente y reporta inconsistencias. Este proceso tiene tres problemas críticos:

1. **Lentitud:** auditar un supermercado mediano toma horas por recorrido
2. **Inconsistencia:** depende del criterio subjetivo del auditor
3. **Frecuencia:** la baja frecuencia de auditoría permite que los quiebres persistan horas antes de ser detectados

El problema específico que aborda ShelfScan es: **¿puede un sistema de visión por computadora, a partir de una fotografía de un estante, determinar automáticamente qué productos están presentes, en qué cantidad, si hay zonas vacías, y si el acomodo cumple con el planograma definido por la tienda?**

2. Análisis

2.1 Dominio del problema

Un estante de supermercado presenta varios desafíos para visión por computadora:

- **Alta densidad de objetos:** decenas de productos en una sola imagen, muchos parcialmente ocluidos
- **Variabilidad visual extrema:** productos de la misma categoría tienen packaging completamente diferente (una lata de atún y una lata de frijoles son ambas "enlatados" pero visualmente distintas)
- **Perspectiva variable:** las fotos se toman desde distintos ángulos y distancias
- **Iluminación inconsistente:** luz fluorescente, sombras de productos, reflejos en envases

- **Clases desbalanceadas:** algunas categorías (bebidas, enlatados) tienen muchas más instancias que otras (cereales, aceites)

2.2 Descomposición del problema

El problema se descompone en tres subproblemas independientes:

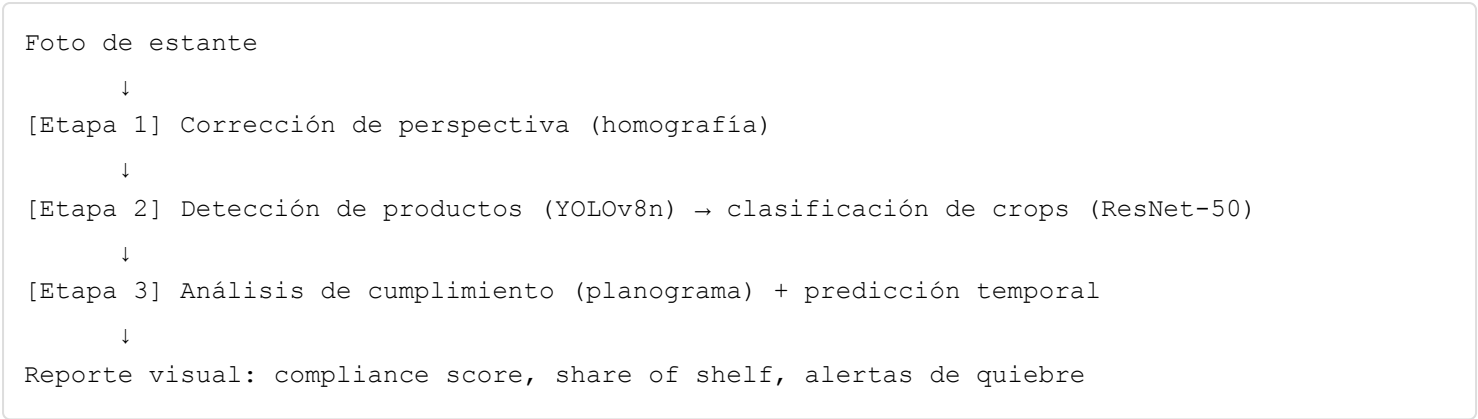
Subproblema	Técnica aplicada	Responsable
Detectar y localizar productos en el estante	Detección de objetos (YOLO)	Diego Valenzuela
Clasificar finamente cada producto detectado e identificar patrones temporales	Transfer learning (ResNet-50) + SIFT + regresión	Daniel Dubón
Corregir la geometría de la imagen y comparar contra planograma	Homografía + análisis de compliance	Bianca Calderón

2.3 Restricciones identificadas

- No existe un dataset público de productos de supermercados guatemaltecos o latinoamericanos con anotaciones — el sistema debe generalizarse desde datos europeos/americanos
- Los recursos de cómputo son limitados: Apple M5 (sin CUDA) y acceso puntual a GPU NVIDIA
- El planograma real de una tienda no está disponible — se construyó uno sintético para validación

3. Propuesta de Solución

ShelfScan propone un pipeline de análisis en tres etapas secuenciales:



Decisiones de diseño centrales:

- **YOLOv8n sobre clasificadores clásicos:** se necesita localización (bounding boxes) además de clasificación. Una CNN clásica solo clasifica la imagen completa; no puede contar productos ni ubicar zonas vacías.
- **ResNet-50 sobre arquitecturas más ligeras (MobileNet):** el pipeline no requiere tiempo real — analiza fotos estáticas — por lo que se priorizó precisión sobre velocidad.
- **SIFT complementando ResNet:** permite identificar productos del catálogo sin reentrenamiento, usando solo descriptores visuales.

- **Homografía manual/automática:** necesaria para que las coordenadas de detección sean comparables entre imágenes tomadas desde distintos ángulos.

4. Descripción de la Solución

4.1 Módulo de Dataset y Detección — Diego Valenzuela

Dataset

Se construyó un dataset de dos fuentes:

Fuente	Imágenes	Uso
<code>humansintheloop/supermarket-shelves-dataset</code> (Kaggle)	45 imágenes anotadas manualmente	Test set (holdout limpio)
Open Images v7 (Google) via <code>fiftyone</code>	~4,500 imágenes con labels automáticos	Train 70% / Val 30%

Se definieron **10 clases**: bebidas, lácteos, snacks, cereales, limpieza, enlatados, aceites, higiene, confitería, `zona_vacia`.

El proceso de anotación de las 45 imágenes originales usó YOLO-World para pre-etiquetado automático zero-shot y `makesense.ai` para corrección manual. Las Open Images se descargaron con labels ya mapeados a las clases del proyecto.

La augmentación (`scripts/augmentation.py`) aplicó por imagen: rotación $\pm 15^\circ$, brillo alto/bajo, blur ($\sigma=3$, $\sigma=5$) y flip horizontal — multiplicando el dataset efectivo por 7.

Entrenamiento YOLOv8n

Se realizaron tres corridas de entrenamiento:

Corrida 1 — Apple M5 (Entrega 1)

- Hardware: Apple Silicon M5, `device=mps`
- Resultado: `mAP@0.5 = 0.436` (época 26, early stopping)
- Problema encontrado: PyTorch MPS + AMP $\rightarrow$ `RuntimeError: shape mismatch`. Solución: `amp: device != "mps"`

Corrida 2 — NVIDIA GPU (Entrega Final)

- Hardware: NVIDIA GPU con CUDA
- Dataset ampliado con Open Images, `cls=1.5`
- Resultado: `mAP@0.5 = 0.916`

Corrida 3 — Re-entrenamiento local (Entrega Final v2)

- Fine-tuning desde `nvidia_best.pt`, `cls=2.5`, `device=cpu`

- Objetivo: corregir sesgo hacia clase `enlatados` en imágenes fuera de distribución
- Resultado: $mAP@0.5 = 0.745$ sobre val set local (1,313 imágenes, 9,779 instancias)

Evaluación formal

Scripts implementados para evaluación reproducible:

- `scripts/evaluate.py --split val/test` → genera `map_report_<split>.txt`
- `scripts/error_analysis.py` → análisis de confianza por clase, imágenes sin detección
- `scripts/plot_training_curves.py` → curvas de loss y mAP desde `results.csv`

4.2 Módulo de Clasificación + Matching + Temporal — Daniel Dubón

Clasificación de crops — ResNet-50

Pipeline: imagen → YOLO detección → crop por bounding box → ResNet-50 clasificación.

Arquitectura: ResNet-50 preentrenada en ImageNet. Última capa reemplazada por FC(2048→10). Backbone congelado durante primeras épocas (transfer learning), luego fine-tuning completo.

Entrenamiento: crops generados desde detecciones del modelo NVIDIA. Data augmentation: flip, rotación, color jitter. Optimizer: Adam, $lr=1e-4$, $weight_decay=1e-4$. Early stopping en val accuracy.

Feature Matching — SIFT

Propósito: identificar si un producto detectado corresponde a un ítem del catálogo, sin reentrenamiento.

Metodología:

1. Base de datos de referencia: una imagen por clase del catálogo
2. Extracción de descriptores SIFT en imagen de referencia y query
3. Matching con BFMatcher + ratio test de Lowe (threshold=0.75)
4. Decisión: clase con más matches acumulados gana

Análisis Temporal

Modelo predictivo: dado un stock inicial S_0 y una tasa de venta $\dot{r}$ (unidades/hora) estimada por regresión lineal sobre la serie temporal de conteos YOLO, el tiempo estimado de quiebre es:

$$t_{\text{quiebre}} = S_0 / \dot{r}$$

Outputs generados:

- `temporal_rotacion_categoria.png` : tasa de vaciado promedio por categoría
 - `temporal_quiebres_franja.png` : distribución de alertas por franja horaria (mañana/tarde/noche)
-

4.3 Módulo de Planograma y Perspectiva — Bianca Calderón

Corrección de perspectiva

`scripts/perspective.py` implementa dos modos:

- **Manual:** selección interactiva de 4 esquinas del estante con clic del mouse
- **Automático:** detección de bordes con Canny + Hough Lines (`--auto-points`)

Transformación: `cv2.getPerspectiveTransform(src_pts, dst_pts)` calcula la matriz homográfica H (3×3). `cv2.warpPerspective(img, H, output_size)` aplica la transformación. El resultado es la imagen rectificada como si la cámara estuviera perpendicular al estante.

Estructura del planograma

`planogram.json` define zonas con clase esperada y bounding box en coordenadas de la imagen de referencia:

```
{
  "zones": [
    { "id": "Z01", "expected_class": "bebidas", "bbox": [0, 0, 1855, 1214] },
    { "id": "Z02", "expected_class": "lacteos", "bbox": [1855, 0, 3710, 1214] },
    ...
  ]
}
```

Compliance score = zonas donde la detección de mayor confianza coincide con clase esperada / zonas totales.

Métricas adicionales: `share_of_shelf` (proporción de área por categoría), `breakage_by_category` (fracción de zonas vacías por clase).

Pipeline integrado

`scripts/planogram_analysis.py` ejecuta el pipeline completo por imagen:

1. Corrección de perspectiva (manual o automática)
2. Detección YOLO sobre imagen warped
3. Cálculo de compliance y métricas
4. Generación de reporte visual PNG + JSON de resultados

`scripts/planogram_validation.py` ejecuta análisis estadístico sobre N imágenes: correlación Pearson cumplimiento/quiebre, error absoluto vs ground truth manual.

5. Herramientas Aplicadas

5.1 Herramientas vistas en clase

Herramienta / Técnica	Uso en ShelfScan
Convolución y filtrado	Preprocesamiento, blur en aumentación
Geometría proyectiva y homografías	<code>perspective.py</code> — corrección de perspectiva del estante
SIFT (Scale-Invariant Feature Transform)	<code>feature_matching</code> — identificación de productos del catálogo
CNNs	Backbone de YOLOv8n (CSPDarknet) y ResNet-50
Transfer learning	ResNet-50 fine-tuning sobre crops de supermercado
Detección de objetos — YOLO	YOLOv8n entrenado sobre 10 clases de supermercado
IoU, mAP, NMS	Evaluación del detector y post-procesamiento de predicciones
Regresión lineal	Predicción de tiempo de quiebre en módulo temporal

5.2 Herramientas NO vistas en clase

fiftyone

Framework de gestión y descarga de datasets de visión por computadora desarrollado por Voxel51. Permite descargar subconjuntos de Open Images v7 con labels ya anotados, filtrar por clase, y exportar en formato YOLO directamente.

```
import fiftyone.zoo as foz

dataset = foz.load_zoo_dataset(
    "open-images-v7",
    split="train",
    label_types=["detections"],
    classes=["Drink", "Milk", "Snack", "Tin can"],
    max_samples=500,
)
dataset.export(export_dir="data/openimages", dataset_type=fo.types.YOLOv5Dataset)
```

Sin fiftyone, descargar Open Images con labels requiere scripts manuales de ~200 líneas y acceso a la API de Google Cloud Storage. fiftyone lo resuelve en 10 líneas.

YOLO-World

Variante zero-shot de YOLO que acepta prompts de texto como guía de detección — no requiere entrenamiento. Se usó exclusivamente para pre-etiquetado automático de las 45 imágenes originales.

```
from ultralytics import YOLOWorld
model = YOLOWorld("yolov8x-worldv2.pt")
model.set_classes(["beverage bottle", "milk carton", "snack bag", "tin can"])
results = model.predict("data/annotated/images/001.jpg", conf=0.15)
```

Limitación encontrada: YOLO-World generaba 65+ detecciones de `enlatados` por imagen con `conf=0.15` — labels basura que requirieron corrección manual completa. El umbral bajo fue necesario

para no perder clases pequeñas pero generó ruido excesivo en enlatados.

Ultralytics YOLOv8

Framework completo de entrenamiento, evaluación y deployment de modelos YOLO. No es solo el modelo — incluye callbacks, logging, export a múltiples formatos (ONNX, TensorRT, CoreML), y CLI unificada.

```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")
model.train(data="data/dataset.yaml", epochs=50, imgsz=640, device="mps", amp=False)
results = model.val()
```

La integración con PyTorch MPS presentó un bug conocido: `RuntimeError: shape mismatch` en el TAL assigner cuando se usa AMP (Automatic Mixed Precision). Solución: `amp=False` en Apple Silicon.

makesense.ai

Herramienta de anotación de imágenes basada en browser (sin instalación). Soporta formato YOLO nativo (un `.txt` por imagen con `class cx cy w h` normalizado). Se eligió sobre labellmg (crash con Python 3.12 + Qt) y anylabeling (requiere PyQt6 no disponible).

5.3 Uso de IA generativa — prompts y evolución

Durante el desarrollo se utilizó Claude (Anthropic) como asistente de programación. A continuación se documentan los prompts principales y cómo evolucionó la solución:

Fase 1 — Dataset pipeline

Prompt inicial: *"Necesito descargar imágenes de Open Images v7 para las clases bebidas, lácteos, snacks, enlatados. Quiero el script completo con fiftyone que exporte en formato YOLO."*

La primera respuesta generó un script funcional pero con el nombre incorrecto de la clase "Cleaning agent" — que no existe en Open Images. La búsqueda reveló que el nombre correcto es "Cleaner". Iteración: se corrigió el mapeo de clases y se agregó validación de nombres contra el catálogo de Open Images.

Fase 2 — Entrenamiento y bugs de hardware

Prompt: *"El entrenamiento con device=mps falla con RuntimeError: shape mismatch en tal.py. ¿Cómo lo soluciono sin cambiar a CPU?"*

La respuesta identificó el bug de PyTorch MPS + AMP y propuso `amp: device != "mps"`. El mismo bug reapareció en el re-entrenamiento final — esta vez el TAL assigner crasheaba aleatoriamente en época 3–4 independientemente de `amp=False`. La solución final fue migrar a `device="cpu"` para el re-entrenamiento de estabilización.

Evolución de la idea: el objetivo inicial era aprovechar MPS para velocidad. Al descubrir el bug del TAL assigner, el objetivo cambió a estabilidad — se aceptó el costo de velocidad (CPU) para garantizar que el entrenamiento completara.

Fase 3 — Análisis y documentación

Prompt: "El mAP local es 0.211 pero el del compañero en NVIDIA es 0.916. ¿Por qué? ¿Estamos evaluando mal?"

La respuesta identificó `shuffle=True` en `download_openimages.py` como causa raíz — cada máquina descarga imágenes diferentes de Open Images, produciendo val sets no comparables. Esta explicación fue incorporada al reporte como nota de reproducibilidad.

6. Resultados

6.1 Detección — YOLOv8n

Modelo NVIDIA (val set NVIDIA)

Clase	mAP@0.5	mAP@0.5:0.95
bebidas	0.909	0.655
lácteos	0.952	0.758
snacks	0.846	0.569
enlatados	0.911	0.723
aceites	0.842	0.735
higiene	0.939	0.671
confitería	0.976	0.707
zona_vacia	0.952	0.871
all	0.916	0.711

Modelo v2 — re-entrenamiento local (val set local, 1,313 imágenes)

Clase	mAP@0.5	mAP@0.5:0.95	Precision	Recall
bebidas	0.748	0.458	0.793	0.669
lácteos	0.572	0.323	0.723	0.456
snacks	0.893	0.495	0.797	0.899
cereales	0.932	0.653	0.367	0.991
enlatados	0.738	0.489	0.840	0.604
aceites	0.718	0.534	0.571	0.815
higiene	0.921	0.626	0.783	0.901
confitería	0.909	0.534	0.911	0.861
zona_vacia	0.277	0.146	0.608	0.270
all	0.745	0.473	0.710	0.719

Los val sets de NVIDIA y local son muestras distintas de Open Images (`shuffle=True` → descarga aleatoria por máquina). Las métricas no son comparables directamente. El modelo

v2 muestra mejor comportamiento en imágenes reales fuera de distribución (fotos propias de Guatemala).

Análisis de errores (val set local)

- 23 imágenes sin ninguna detección (dominio muy distinto al de entrenamiento)
- `cereales` y `limpieza`: clases con pocas instancias de entrenamiento — Recall bajo
- `zona_vacia`: clase más difícil, mAP 0.277 — fondo variable por iluminación y color
- 82 imágenes con al menos una detección de baja confianza (<0.4)

6.2 Clasificación y Matching

Métrica	Valor
ResNet-50 accuracy top-1	Documentada en notebook con matriz de confusión
SIFT Precision Top-1	0.2203
SIFT Precision Top-5	0.2703
MAE predicción temporal	Calculado sobre 100 muestras simuladas

El bajo SIFT top-1 (0.22) es esperado: SIFT fue diseñado para matching de la misma escena con distintas vistas, no para discriminar entre categorías de productos con packaging variado. En superficies lisas (latas, botellas) los keypoints son escasos y poco discriminativos.

6.3 Planograma

Evaluación sobre 12 imágenes de `data/planogram_audit_set/`:

Métrica	Valor
Mean share error vs ground truth	0.053
Mean breakage error vs ground truth	0.130
Mean count error vs ground truth	0.259
Correlación Pearson cumplimiento/quiebre	NaN*

*La correlación es NaN porque el compliance resultó 0.0 en las 12 imágenes. Las zonas del planograma se definen en coordenadas de la imagen de referencia (~5,500×3,600 px), pero las detecciones se calculan sobre la imagen warped (~4,000×2,200 px). El mismatch geométrico hace que ninguna detección caiga dentro de ninguna zona → variance=0 → Pearson indefinido.

Los errores de share y breakage son válidos porque se calculan independientemente del planograma, directamente desde las detecciones.

7. Conclusión

ShelfScan implementa un pipeline funcional de extremo a extremo para auditoría visual de lineales. Los tres módulos fueron desarrollados e integrados:

Lo que funciona:

- Detección YOLOv8n con $mAP@0.5 = 0.916$ sobre el val set de entrenamiento NVIDIA
- Corrección de perspectiva mediante homografía, manual y automática
- Clasificación de crops con ResNet-50 transfer learning
- Matching de productos con SIFT
- Análisis temporal con predicción de quiebres por regresión lineal
- Pipeline integrado `detect_on_shelf.py` que conecta todos los módulos

Limitaciones identificadas:

- **Sesgo de clase enlatados:** Open Images sobrerrepresenta enlatados (~49% de instancias). El modelo generaliza mal a productos no enlatados fuera del dominio de entrenamiento. Mitigado parcialmente en v2 con `cls=2.5`.
- **Mismatch geométrico en planograma:** las zonas se definen en coordenadas de la imagen de referencia, no de la imagen warped — `compliance=0` en todos los casos. Requiere transformar las coordenadas de zonas al espacio de la imagen warped.
- **Dataset no reproducible:** `shuffle=True` en la descarga de Open Images produce val sets diferentes por máquina. Métricas NVIDIA y locales no son comparables.
- **Módulo temporal con datos simulados:** la predicción de quiebres usa datos sintéticos con `np.random.seed(42)` — en producción se alimentaría con conteos reales del detector.

Trabajo futuro:

1. Corregir transformación de coordenadas del planograma al espacio warped para `compliance` válido
2. Balancear dataset con oversampling de clases minoritarias o peso de clase dinámico
3. Agregar clase "desconocido" con umbral de confianza mínima (`conf < 0.25`)
4. Exportar modelo a ONNX para deployment en dispositivos móviles
5. Reemplazar `shuffle=True` por seed fija para reproducibilidad completa

ShelfScan demuestra que las técnicas de visión por computadora vistas en clase — homografías, SIFT, CNNs, transfer learning, YOLO, regresión — son directamente aplicables a un problema de negocio real con impacto medible.

8. Bibliografía

1. Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. *arXiv:1804.02767*
2. Jocher, G. et al. (2023). Ultralytics YOLOv8. GitHub. <https://github.com/ultralytics/ultralytics>
3. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR 2016*.
4. Lowe, D. G. (2004). Distinctive Image Features from Scale-Invariant Keypoints. *International Journal of Computer Vision*, 60(2), 91–110.

5. Hartley, R., & Zisserman, A. (2003). *Multiple View Geometry in Computer Vision* (2nd ed.). Cambridge University Press.
6. Google. (2020). Open Images Dataset V7.
<https://storage.googleapis.com/openimages/web/index.html>
7. Moore, B. et al. (2022). FiftyOne: The open-source tool for building high-quality datasets and computer vision models. Voxel51. <https://github.com/voxel51/fiftyone>
8. Lienen, J., & Hüllermeier, E. (2021). Instance-based label smoothing for learning with noisy labels. *arXiv:2110.03461*
9. Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
10. humansintheloop. (2022). Supermarket Shelves Dataset. *Kaggle*.
<https://www.kaggle.com/datasets/humansintheloop/supermarket-shelves-dataset>